

MITS 6700G — Final Project Report

Two-Layer Hybrid CVE Network Analysis: Anchored on Log4Shell (CVE-2021-44228)

Group 8 | Network Science and Graph Analytics

Table of Contents

1. [Project Overview](#)
 2. [Dataset & Data Collection](#)
 3. [Methodology — Two-Layer Hybrid Approach](#)
 4. [Step 1: Graph Construction Results](#)
 5. [Step 2: Degree Distribution & Scale-Free Analysis](#)
 6. [Step 3: Centrality Analysis](#)
 7. [Step 4: Community Detection](#)
 8. [Step 5: SIR Epidemic Simulation](#)
 9. [Step 6: Robustness Analysis](#)
 10. [Step 7: Summary Dashboard](#)
 11. [All Output Files Reference](#)
 12. [Conclusions & Recommendations](#)
-

1. Project Overview

What We Did

We built a **network science analysis** of cybersecurity vulnerabilities from the NVD (National Vulnerability Database) 2021 dataset, using a "Two-Layer Hybrid" graph model with **Log4Shell (CVE-2021-44228)** as the anchor node.

The goal was to answer: *How central, how influential, and how dangerous is Log4Shell compared to all other vulnerabilities of 2021?* We used graph theory and network science measures to answer this quantitatively.

Why This Approach?

Traditional vulnerability analysis looks at each CVE in isolation (CVSS score, severity tag). Our approach treats vulnerabilities as a **connected network** — two CVEs are linked if they affect the same software product. This reveals hidden structural patterns: which CVE is a network "hub," which communities form around shared attack surfaces, and how a vulnerability outbreak propagates through the network.

The 8 Design Fixes Applied

During implementation, we applied 8 critical project-specific fixes:

Fix	Description
FIX-1	k=1000 fixed pivots for betweenness (approximate, 3.2% error)
FIX-2	N defined first in SIR simulation; included in return dict
FIX-3	SIR plot uses result['N'] (not result['s_curve'][0])
FIX-4	LCC (Largest Connected Component) extracted after projection
FIX-5	Ego-graph built from LCC, not raw bipartite
FIX-6	Log4Shell ego-graph radius = 2 hops (not 1)
FIX-7	compare_beta_values() uses result['N'] for fraction infected
FIX-8	Betweenness normalization uses $n*(n-1)$ for undirected graphs

2. Dataset & Data Collection

Source

- **Provider:** fkcie-cad/nvd-json-data-feeds (Fraunhofer FKIE Institute)
- **Format:** NVD API 2.0 JSON schema
- **Year:** 2021 (all CVEs published between 2021-01-01 and 2021-12-31)
- **File:** `nvdCVE-1.1-2021.json` (215.7 MB decompressed)
- **Checksum:** SHA-256 verified at download

Dataset Statistics

Metric	Value
Total CVEs in 2021	23,437
CVEs with CVSS score	22,567
CVEs with no CPE data	1,120
Critical (CVSS ≥ 9.0)	2,693
High (CVSS 7.0–8.9)	9,489
Medium (CVSS 4.0–6.9)	9,633
Low (CVSS < 4.0)	1,622

Log4Shell Verification

```
CVE ID      : CVE-2021-44228
CVSS Score  : 10.0 (maximum possible)
Severity    : CRITICAL
Products    : 381 affected software products/versions
Published   : 2021-12-10
```

3. Methodology — Two-Layer Hybrid Approach

Why Two Layers?

A single-layer analysis would either be too broad (all 23K CVEs) or too narrow (just Log4Shell). The Two-Layer Hybrid gives us:

- **Layer 1 (Global):** The "big picture" — all serious CVEs, to find Log4Shell's position globally
- **Layer 2 (Local):** The "neighborhood picture" — only CVEs directly or closely related to Log4Shell

Layer 1 — Global Graph Construction

Filter rule: Keep only CVEs with CVSS ≥ 7.0 AND at least 1 CPE (affected product)

- **Why CVSS ≥ 7.0 ?** Focus on High and Critical severity — the ones that actually matter for defenders. Medium/Low vulnerabilities add noise.
- **Why require CPE?** CPE (Common Platform Enumeration) is the product identifier that creates edges between CVEs. Without CPE, a CVE cannot be linked to others.

Graph type: CVE-CVE projection via shared CPE products

- We start with a **bipartite graph**: CVE nodes on one side, CPE product nodes on the other. A CVE is connected to each CPE it affects.
- Then we **project** onto CVEs: if two CVEs share a CPE (same product), they get an edge.
- We extract the **Largest Connected Component (LCC)** — the main connected region of the network.

Layer 2 — Local Ego-Graph Construction

Filter rule: All CVEs within 2 hops (graph distance ≤ 2) of Log4Shell in Layer 1

- **Why 2 hops?** 1-hop gives only direct neighbors (same product). 2-hop includes "attack surface cousins" — CVEs that share a product with something that also affects Log4Shell's products. This captures the broader risk neighborhood.
- This gives 2,942 nodes — all CVEs with meaningful proximity to Log4Shell.

Tools Used

Library	Version	Purpose
NetworkX	3.6.1	Graph construction, LCC, PageRank, degree centrality
igraph	1.0.0	Betweenness, closeness, eigenvector (C backend — 10–100× faster)
python-louvain	0.16	Community detection (Louvain algorithm)
powerlaw	2.0.0	Power-law exponent fitting for degree distributions
pandas	2.1.3	Data frames and CSV export
matplotlib / seaborn	3.8.2 / 0.13.0	Publication-quality visualizations

4. Step 1: Graph Construction Results

Layer 1 — Global Graph

Property	Value	Interpretation
CVE nodes in bipartite	12,013	Filtered from 23,437 total
CPE product nodes	36,593	Unique affected products
Bipartite edges	156,526	CVE-to-product links
CVE-CVE projection edges	706,044	Pairs sharing a product
LCC nodes (N)	3,944	32.8% of projection — main cluster
LCC edges (M)	628,178	Very dense
Graph density	0.0808	~8% of all possible edges exist
Average degree	318.55	Each CVE shares products with ~319 others
Max degree	1,374 (CVE-2021-37969)	Most-connected CVE
Components	1	Fully connected — one giant cluster

Layer 2 — Local Ego-Graph

Property	Value	Interpretation
Nodes (N)	2,942	CVEs within 2 hops of Log4Shell
Edges (M)	545,367	Very dense neighborhood
Graph density	0.1261	Even denser than Layer 1 (Log4Shell's neighborhood is tightly coupled)
Average degree	370.75	Each node connects to ~371 others on average
Max degree	1,374	Same max-degree node (CVE-2021-37969) — it's inside Log4Shell's neighborhood
Components	1	Fully connected

What This Means

The network is **extremely dense**. An average CVE shares at least one affected product with 318 other CVEs. This means the attack surface is massively interconnected — a system vulnerable to one CVE is statistically likely vulnerable to hundreds more in the same product family. Log4Shell's 2-hop neighborhood captures 74.6% of the entire Layer 1 network (2942 of 3944), confirming its position near the center of the vulnerability universe.

5. Step 2: Degree Distribution & Scale-Free Analysis

Output Files

- `degree_dist_global.png` — Global Layer 1 degree distribution (log-log power-law fit)
- `degree_dist_local.png` — Local Layer 2 degree distribution (log-log power-law fit)

What These Plots Show

Each plot has two panels:

1. **Left (linear scale):** Histogram of degree values. Shows a sharp right skew — most nodes have moderate degree, a few have very high degree.
2. **Right (log-log scale):** The same data on logarithmic axes. A power-law distribution appears as a straight line on a log-log plot. Both plots show this linear relationship, confirming scale-free behavior.

Results

Network	Power-law exponent γ	Scale-free?
Layer 1 (Global)	2.4069	✓ YES ($2.0 < \gamma < 3.5$)
Layer 2 (Local)	2.0924	✓ YES (γ near 2.0 = stronger hubs)

Why This Matters — Scale-Free Networks Explained

A scale-free network (γ between 2 and 3.5) means:

- **Most nodes have few connections** (most CVEs affect only a few products)
- **A few "hub" nodes have extremely many connections** (a few CVEs affect hundreds of products)
- **This is not random** — it follows a power law $P(k) \sim k^{-\gamma}$

Practical implication for cybersecurity: Scale-free networks are:

- **Resilient to random failures** — if you randomly patch CVEs, you'll mostly hit low-degree CVEs and the network stays connected
- **Vulnerable to targeted attacks** — if attackers focus on hub CVEs (high-degree), the network fragments quickly

Log4Shell affects 381 products — well above the average of 13 products per CVE. It's a hub node.

The local layer's lower $\gamma = 2.0924$ means Log4Shell's neighborhood has even more extreme hub concentration than the global network.

6. Step 3: Centrality Analysis

What Is Centrality?

Centrality measures tell us how "important" a node is in a network, from different perspectives:

Measure	What It Asks	Cybersecurity Meaning
Degree Centrality	How many direct connections?	How many other CVEs share a product with this CVE?
Betweenness Centrality	How often on shortest paths between others?	Is this CVE a "bridge" that connects different vulnerability clusters?
Closeness Centrality	How close to all other nodes on average?	How quickly can this CVE "reach" all others through shared products?
PageRank	Which nodes are linked to by other important nodes?	Is this CVE referenced/linked by high-severity, well-connected CVEs?
Eigenvector Centrality	Is this node connected to other important nodes?	Does this CVE affect products that themselves are affected by many other CVEs?

Implementation Notes

- **Betweenness (Layer 1):** Approximate with k=1000 random pivot nodes. Error = $1/\sqrt{1000} = 3.16\%$. Used igraph C backend.
- **Betweenness (Layer 2):** Exact (all nodes), using igraph C backend.
- **Closeness:** Exact using igraph C backend.
- **Eigenvector:** igraph C backend (NetworkX would fail to converge on this dense graph).
- **PageRank:** NetworkX 3.6.1, damping factor 0.85, 1000 iterations.

Log4Shell Centrality Report

LOG4SHELL CENTRALITY REPORT – CVE-2021-44228			
Measure	Score	Rank	Top %
Degree (Global)	0.24093	40	1.0%
Betweenness(Glob)	0.07540	4	0.1%
Closeness(Glob)	0.48553	12	0.3%
PageRank(Global)	0.00141	1	0.0%
Degree (Local)	0.32302	40	1.4%
Betweenness(Loc)	0.08110	2	0.1%
Eigenvector(Loc)	0.59631	10	0.3%
PageRank(Local)	0.04374	1	0.0%

Global Top-3 PageRank

Rank	CVE	PageRank	CVSS
1	CVE-2021-44228 (Log4Shell)	0.001407	10.0
2	CVE-2021-22946	0.000748	7.5

Rank	CVE	PageRank	CVSS
3	CVE-2021-3450	0.000723	7.4

Interpretation of Each Measure

Degree Centrality (Rank #40 globally, top 1.0%)

- Score 0.241 means Log4Shell is directly connected to 24.1% of all CVEs in the network
- Its 381 affected products create edges to ~940 other CVEs directly
- It's a hub, but not THE most-connected node — CVE-2021-37969 has degree 1374 vs Log4Shell's ~940

Betweenness Centrality (Rank #4 globally, top 0.1%) — VERY HIGH

- Score 0.0754 means ~7.5% of all shortest paths between any two CVEs pass THROUGH Log4Shell
- This means Log4Shell is a critical **bridge** in the vulnerability network
- If you want to understand the "attack chain" from one vulnerability to another, Log4Shell is frequently on that chain
- Rank #4 of 3944: only 3 CVEs serve as better bridges

Closeness Centrality (Rank #12 globally, top 0.3%)

- Score 0.486 means Log4Shell can reach any other CVE in the network in ~2 steps on average
- High closeness = Log4Shell is "near the center" of the network — its product ecosystem overlaps with nearly everything
- This explains why it spread so quickly: it was already 2 steps from almost every other vulnerable system

PageRank (Rank #1 globally and locally — THE MOST IMPORTANT FINDING)

- **Log4Shell is the #1 most important CVE of 2021 by PageRank in both layers**
- PageRank (borrowed from Google's web ranking) assigns importance not just based on how many connections you have, but based on whether your connections are themselves important
- Score 0.001407 is 1.88× higher than the #2 CVE — a significant gap
- **Interpretation:** Log4Shell affects products that are themselves widely used, broadly connected, and highly critical. The Apache Log4j library's universal presence in Java software means that connecting to Log4Shell means connecting to the epicenter of enterprise software.

Output Files for This Section

- **centrality_radar_log4shell.png** — Spider/radar chart showing all 8 centrality scores normalized. The "shape" of this chart shows Log4Shell's centrality profile at a glance. A large PageRank arm with smaller degree arm is characteristic of a "hidden influencer" node — well-connected to important nodes but not necessarily the highest-degree node.
- **top10_pagerank_global.png** — Top 10 CVEs by PageRank in Layer 1. Log4Shell bar is longest.
- **top10_betweenness_global.png** — Top 10 by betweenness in Layer 1. Log4Shell is #4.
- **top10_closeness_global.png** — Top 10 by closeness in Layer 1. Log4Shell is #12.
- **top10_degree_cent_global.png** — Top 10 by degree centrality in Layer 1. Log4Shell is #40.
- **top10_pagerank_local.png** — Top 10 CVEs by PageRank in Layer 2 (Log4Shell neighborhood). Log4Shell is #1.
- **top10_betweenness_local.png** — Top 10 by betweenness in Layer 2. Log4Shell is #2.

- **top10_closeness_local.png** — Top 10 by closeness in Layer 2.
- **top10_degree_cent_local.png** — Top 10 by degree in Layer 2.
- **centrality_global.csv** / **centrality_local.csv** — Full CSV with all 5 centrality scores for all nodes, sortable in Excel.

7. Step 4: Community Detection

What Is Community Detection?

Community detection finds groups of nodes that are more densely connected to each other than to the rest of the network. In our CVE network, a community = a cluster of vulnerabilities that share many of the same affected products (same ecosystem, same software family, same attack surface).

Algorithm: Louvain Method

The Louvain algorithm is a widely-used greedy modularity-maximization method:

1. Each node starts in its own community
2. Iteratively, nodes are moved to the neighboring community that most increases modularity
3. Communities are merged, and the process repeats at a coarser level
4. Stops when no improvement is possible

Why Louvain? It's fast ($O(N \log N)$), produces high-quality communities, and doesn't require specifying the number of communities in advance.

Results

Metric	Value	Interpretation
Number of communities	49	49 distinct vulnerability ecosystems
Modularity Q	0.6703	Excellent ($Q > 0.6$ = strong community structure)
Largest community	1,091 nodes	~27.7% of all CVEs
Smallest community	2 nodes	Isolated pairs

Top 5 Communities by Size

Rank	Community ID	Size
1	4 (Log4Shell's community)	1,091
2	1	749
3	6	406
4	2	312
5	0	300

Log4Shell's Community (Community #4)

Property	Value
Community ID	4
Size	1,091 CVEs
Mean CVSS	8.26
% Critical	14.0% (153 critical CVEs)

Top 10 CVEs by CVSS in Log4Shell's Community:

Rank	CVE ID	CVSS	Severity
1	CVE-2021-31891	10.0	CRITICAL
2	CVE-2021-41556	10.0	CRITICAL
3	CVE-2021-38503	10.0	CRITICAL
4	CVE-2021-44228	10.0	CRITICAL
5	CVE-2021-3781	9.9	CRITICAL
6	CVE-2021-20314	9.8	CRITICAL
7	CVE-2021-28834	9.8	CRITICAL
8	CVE-2021-44732	9.8	CRITICAL
9	CVE-2021-43299	9.8	CRITICAL
10	CVE-2021-43113	9.8	CRITICAL

Interpretation

- Log4Shell belongs to the **largest community of 2021** — 1,091 CVEs share overlapping product ecosystems
- The mean CVSS of 8.26 in this community is **above the dataset average** — this is a high-severity cluster
- 14% Critical rate means if an organization is exposed to Log4Shell, they are statistically likely exposed to 152 other critical CVEs in the same infrastructure
- **High modularity Q = 0.6703** confirms that the 49 communities are real clusters, not random groupings. This is strong community structure.

Output Files

- **community_size_dist.png** — Bar chart of community sizes (sorted). Shows the long tail of 49 communities, with a few large communities and many small ones. This is the classic "power-law community size" pattern common in real networks.
- **community_network_plot.png** — Network visualization with 600 sampled nodes, colored by community. Use this to show your team the visual structure: you'll see distinct color clusters with Log4Shell (highlighted separately) visible in the large red/central cluster.

- [log4shell_community_members.csv](#) — Full list of all 1,091 CVEs in Log4Shell's community with CVSS scores and severity. Useful for risk prioritization.

8. Step 5: SIR Epidemic Simulation

What Is the SIR Model?

The SIR (Susceptible-Infected-Recovered) model is a mathematical model of outbreak dynamics, originally from epidemiology. We adapt it to cyber-vulnerability spread:

State	Original Meaning	Our Meaning
S (Susceptible)	Healthy, can get infected	CVE not yet exploited in a system
I (Infected)	Currently infected	CVE is actively being exploited
R (Recovered)	Immune after infection	CVE has been patched/mitigated

Simulation rule: At each time step, each Infected CVE transmits to each of its Susceptible neighbors with probability β . Each Infected CVE recovers with probability γ .

Why run SIR on a CVE network? If an attacker starts from Log4Shell (initial I node), how quickly does the "attack" spread through the vulnerability network? A path in the CVE network represents: "if you're vulnerable here, you're probably vulnerable there too" — same server, same software stack, same product family.

Parameters

Parameter	Value	Rationale
γ (recovery rate)	0.1	10% chance of patching per time step
β values tested	0.1, 0.2, 0.3, 0.5	Range from slow to fast spread
n_runs	20	Averaged over 20 independent runs for stability
Seed node	CVE-2021-44228	Log4Shell as outbreak origin
Network	Layer 2 (2,942 nodes)	Local neighborhood = realistic attack surface

Results Table

β (spread rate)	Peak Infected	Peak Timestep	Total Infected (R_final)
0.1 (slow)	79.9% (2,352 CVEs)	Step 4	98.5% (2,899 CVEs)
0.2 (medium)	81.9% (2,409 CVEs)	Step 4	99.3% (2,922 CVEs)
0.3 (fast)	84.4% (2,483 CVEs)	Step 3	99.7% (2,933 CVEs)
0.5 (very fast)	86.6% (2,549 CVEs)	Step 3	99.9% (2,938 CVEs)

Critical Findings — What These Numbers Mean

1. Catastrophic Spread Regardless of Transmission Rate Even at $\beta=0.1$ (very low spread probability — 10% chance per neighbor per step), **98.5% of Log4Shell's 2-hop network gets "infected"** by the epidemic end. This is not a property of β — it's a property of the network structure.

2. The Network Density Drives Spread, Not the Transmission Rate Compare: $\beta=0.1 \rightarrow 98.5\%$ vs $\beta=0.5 \rightarrow 99.9\%$. The difference is only 1.4 percentage points. Changing β fivefold barely changes the outcome. Why? Because the Layer 2 graph is so dense (avg degree 370, density 0.126) that once the epidemic starts, there are so many paths between every pair of nodes that the disease will reach almost everyone regardless.

3. Extremely Fast Spread — Peak at Step 3–4 The epidemic peaks in just 3–4 timesteps. In a real cyber context, if 1 timestep $\approx$ 1 day of exploitation activity, Log4Shell's vulnerability ecosystem could see maximum exploitation within **3–4 days** of initial discovery. This matches the historical reality: Log4Shell was disclosed on Dec 10, 2021 and was under active mass exploitation within 72 hours.

4. Basic Reproduction Number R_0 At $\beta=0.1$, $\gamma=0.1$ with avg degree $k=370$: $R_0 = \beta \times k / \gamma = 0.1 \times 370 / 0.1 = 370 \gg 1$. Any $R_0 \gg 1$ guarantees epidemic spread. This network is super-critical for virtually any realistic β value.

Output Files

- **sir_single_beta0.1.png** through **sir_single_beta0.5.png** — Individual SIR curves for each β value. Each plot shows $S(t)$, $I(t)$, $R(t)$ over time. The S curve drops steeply, I curve peaks then falls, R curve rises monotonically. Show these to demonstrate the different speeds of spread.
- **sir_beta_comparison.png** — All 4 β values overlaid on one plot for direct comparison. **USE THIS ONE** in presentations — it clearly shows that all curves converge to near-100% total infection, just at different speeds.
- **hop_reachability.png** — Bar chart showing how many nodes are reachable from Log4Shell at each hop distance (1-hop, 2-hop, 3-hop, etc.). Shows the "exposure explosion" with each hop.

9. Step 6: Robustness Analysis

What Is Robustness Analysis?

Robustness measures how well a network survives when nodes are removed. We simulate two types of attacks:

- **Random attack:** Remove nodes in a uniformly random order (simulates random failures or random patching)
- **Targeted attack:** Remove nodes in order of highest degree first (simulates an intelligent attacker or systematic targeted patching)

Metric: After removing a fraction f of nodes, how large is the Largest Connected Component (LCC) as a fraction of the original? When LCC collapses, the network has "fragmented" — a critical infrastructure concept.

Critical threshold f_c : The fraction of nodes that must be removed to reduce LCC to $< 5\%$ of original size.

Results

Attack Type	Critical Threshold f_c	Interpretation
Random attack	f_c = 0.92	Must remove 92% of CVEs randomly to fragment the network
Targeted attack	f_c = 0.69	Only need to remove 69% of top-degree CVEs to fragment it
More vulnerable to:	TARGETED	✓ Confirms scale-free network property

What Does This Mean?

Random attack (f_c = 0.92) — Extremely Resilient If you randomly patch CVEs, you need to patch 92% of them before the network loses connectivity. This is a near-impossible target for any organization. Random patching barely hurts the network structure.

Targeted attack (f_c = 0.69) — Significantly More Vulnerable If you systematically target high-degree CVEs first (the "hub" CVEs that affect many products), connectivity collapses much sooner. You only need to eliminate the top 69% by degree.

Difference = 0.92 - 0.69 = 0.23: This gap of 23 percentage points is the "targeted attack advantage." This is a classic property of scale-free networks, sometimes called the "Achilles' heel" of the Internet and other real-world networks (first shown by Barabási et al., 2000).

Practical Security Recommendation: This result mathematically justifies **risk-based patching** over random patching. If defenders focus on high-degree CVEs (those affecting many products) first, they can disrupt the vulnerability network much more efficiently than random patching.

Log4Shell's Implications for Targeted Attack: Log4Shell has degree ~940 in Layer 1. When targeted attack proceeds degree-first, Log4Shell would be among the first ~1% of nodes removed. Its removal would disconnect 153 critical-CVE connections in its community, fragmenting the large Community #4 cluster.

Output Files

- **robustness_comparison.png** — Dual-line plot showing LCC size vs. fraction of nodes removed, for both random (blue) and targeted (red) attacks. The two curves diverge visibly: random (gentle decline) vs. targeted (steeper, earlier collapse). **USE THIS PLOT** to argue for targeted patching strategy.

10. Step 7: Summary Dashboard

Output File

- **SUMMARY_DASHBOARD.png** — A 2×3 panel figure combining the 6 most important results in a single publication-ready image:
 1. (Top left) Degree distribution global (power-law)
 2. (Top center) Log4Shell centrality radar chart
 3. (Top right) Community size distribution
 4. (Bottom left) SIR epidemic curve (β=0.3)

5. (Bottom center) Robustness comparison
6. (Bottom right) Community network visualization

When to use this: This is your one-stop-shop figure for presentations. It tells the complete story in one slide. Show this as the "executive summary" figure.

11. All Output Files Reference

Total files generated: **27 files** in the `outputs/` directory.

File	Size	Step	What It Shows	Use In Report?
<code>SUMMARY_DASHBOARD.png</code>	999 KB	7	6-panel summary of all analyses	★ YES — main summary slide
<code>degree_dist_global.png</code>	583 KB	2	Power-law fit $\gamma=2.4069$ (Layer 1)	✓ Yes — scale-free proof
<code>degree_dist_local.png</code>	522 KB	2	Power-law fit $\gamma=2.0924$ (Layer 2)	✓ Yes — scale-free proof
<code>centrality_radar_log4shell.png</code>	519 KB	3	Log4Shell's 8-metric centrality profile	★ YES — centerpiece
<code>top10_pagerank_global.png</code>	363 KB	3	Log4Shell #1 PageRank globally	✓ Yes — headline result
<code>top10_betweenness_global.png</code>	371 KB	3	Log4Shell #4 betweenness	✓ Yes
<code>top10_closeness_global.png</code>	364 KB	3	Log4Shell #12 closeness	Optional
<code>top10_degree_cent_global.png</code>	371 KB	3	Log4Shell #40 degree (top 1%)	Optional
<code>top10_pagerank_local.png</code>	364 KB	3	Log4Shell #1 PageRank in neighborhood	✓ Yes
<code>top10_betweenness_local.png</code>	373 KB	3	Log4Shell #2 betweenness locally	Optional
<code>top10_closeness_local.png</code>	365 KB	3	Log4Shell closeness in neighborhood	Optional
<code>top10_degree_cent_local.png</code>	357 KB	3	Log4Shell degree in neighborhood	Optional
<code>community_network_plot.png</code>	830 KB	4	Network visualization, 600 nodes, colored by community	✓ Yes — visual impact

File	Size	Step	What It Shows	Use In Report?
community_size_dist.png	326 KB	4	49 community sizes distribution	✓ Yes
log4shell_community_members.csv	27 KB	4	All 1,091 CVEs in Log4Shell's community	 Appendix data
sir_beta_comparison.png	409 KB	5	All 4 β SIR curves overlaid	★ YES — key epidemic result
sir_single_beta0.1.png	360 KB	5	SIR S/I/R curves at $\beta=0.1$	Optional detail
sir_single_beta0.2.png	359 KB	5	SIR S/I/R curves at $\beta=0.2$	Optional detail
sir_single_beta0.3.png	361 KB	5	SIR S/I/R curves at $\beta=0.3$	Optional detail
sir_single_beta0.5.png	358 KB	5	SIR S/I/R curves at $\beta=0.5$	Optional detail
hop_reachability.png	371 KB	5	Nodes reachable per hop from Log4Shell	✓ Yes — reach analysis
robustness_comparison.png	457 KB	6	Random vs targeted attack LCC comparison	★ YES — patching strategy
centrality_global.csv	395 KB	3	All centrality scores Layer 1 (3,944 rows)	 Data appendix
centrality_local.csv	351 KB	3	All centrality scores Layer 2 (2,942 rows)	 Data appendix
download_log.txt	4 KB	0	Data download verification log	Reference only
install_log.txt	9 KB	0	Package installation log	Reference only
pipeline_log.txt	3 KB	0	Runtime pipeline log	Reference only

★ Recommended Figures for Presentation (Priority Order)

1. [SUMMARY_DASHBOARD.png](#) — Use as intro/overview slide
2. [centrality_radar_log4shell.png](#) — Central result: Log4Shell's centrality profile
3. [top10_pagerank_global.png](#) — Headline: Log4Shell is #1 PageRank
4. [sir_beta_comparison.png](#) — Epidemic analysis headline
5. [robustness_comparison.png](#) — Security recommendation
6. [community_network_plot.png](#) — Visual impact / community structure
7. [degree_dist_global.png](#) — Scale-free property proof

12. Conclusions & Recommendations

Summary of Key Findings

Finding 1: Log4Shell is the #1 Most Influential Vulnerability of 2021

By PageRank — the most comprehensive measure of network influence — Log4Shell (CVE-2021-44228) ranks **#1 of 3,944** serious CVEs globally, and **#1 of 2,942** in its own neighborhood. No other 2021 CVE has more network influence. This validates the security community's assessment of Log4Shell's severity from a purely data-driven, network-theoretic perspective.

Finding 2: Log4Shell is a Critical Bridge (#4 Betweenness Globally)

Betweenness rank #4 globally means Log4Shell sits on 7.5% of all shortest paths between CVEs. In practical terms, any attack chain or vulnerability assessment that crosses different product ecosystems will likely pass through Log4Shell. It's not just a standalone vulnerability — it's a connectivity hub.

Finding 3: The CVE Network is Scale-Free ($\gamma = 2.41$ and 2.09)

Both network layers exhibit power-law degree distributions with exponents in the range (2.0, 3.5), confirming scale-free topology. This has two practical implications:

- Random patching is inefficient ($f_c = 0.92$ — need to patch 92% for meaningful impact)
- Targeted patching is highly efficient ($f_c = 0.69$ — patch hub CVEs first for maximum impact)

Finding 4: Log4Shell's Community is a High-Severity Cluster of 1,091 CVEs

With mean CVSS 8.26 and 14% Critical CVEs, the 1,091-CVE community surrounding Log4Shell represents the most dangerous concentration of vulnerabilities in 2021. Organizations using Log4j-based software are likely running products covered by this entire high-severity cluster.

Finding 5: Epidemic Spread from Log4Shell is Catastrophic at Any β

SIR simulation shows 98.5%–99.9% of Log4Shell's 2-hop neighborhood becomes "infected" regardless of transmission rate. The network is too dense for β to matter much — the structure itself guarantees near-complete spread. This makes Log4Shell not just dangerous in isolation but dangerous as a propagation origin.

Finding 6: Community Modularity is Strong ($Q = 0.6703$)

A modularity of 0.6703 means the 49 vulnerability communities are real and meaningful, not statistical noise. Organizations can use these communities to identify product-specific risk clusters: if you're exposed to one CVE in Community #4, you should audit your exposure to all 1,091 CVEs in that community.

Recommendations for the Security Team

Recommendation 1: Prioritize Hub CVEs for Patching The robustness analysis proves that targeted (degree-ordered) patching disrupts the vulnerability network 23 percentage points more efficiently than random patching. Use the `centrality_global.csv` file, sort by degree centrality, and prioritize patches accordingly.

Recommendation 2: Treat Community #4 as a Single Risk Unit The 1,091 CVEs in Log4Shell's community share overlapping affected products. An organization that is exposed to Log4Shell is very likely exposed to a large fraction of Community #4. Use `log4shell_community_members.csv` to cross-reference against your asset inventory.

Recommendation 3: Log4Shell Patching Was Urgently Correct The network analysis confirms what the security community declared: Log4Shell is uniquely dangerous. It is #1 PageRank, #4 betweenness, has 381 affected products, belongs to the largest community, and is the origin of near-total epidemic spread in its neighborhood. Any organization that deprioritized Log4Shell patching in December 2021 made a measurable, quantifiable mistake.

Recommendation 4: Monitor High-Betweenness CVEs for Active Exploitation The 3 CVEs ranked above Log4Shell in betweenness (ranks #1–#3) serve as even more critical network bridges. These CVEs may not have the same media attention but represent critical structural chokepoints. They should be treated as high-priority even if their CVSS scores are lower.

Recommendation 5: Don't Rely on CVSS Alone Log4Shell's degree rank is #40 — it is not the most-connected node. CVE-2021-37969 has the highest degree (1374). Yet Log4Shell is #1 by PageRank. This demonstrates that CVSS and even degree alone are insufficient for prioritization. Network position (PageRank, betweenness) provides essential additional signal.

Limitations

1. **Graph density:** The CVE projection graph is extremely dense (8% density vs. ~0.01% for typical social networks), which may be partly due to the bipartite projection method amplifying product co-occurrences. A weighted projection or minimum co-occurrence threshold could produce sparser, more meaningful graphs.
 2. **CPE data completeness:** 1,120 CVEs (4.8%) had no CPE data and were excluded. These may include newly discovered vulnerabilities where product mapping was not yet complete.
 3. **Year boundary:** We only analyze 2021 CVEs. Log4Shell's cross-year impact (exploitation continued in 2022–2024) and its relationship to pre-2021 vulnerabilities in the same ecosystem is not captured.
 4. **Approximate betweenness:** Layer 1 betweenness used $k=1000$ random pivot sampling ($\pm 3.2\%$ error). Exact computation was computationally infeasible on this dense graph in Python, even with igraph's C backend.
 5. **SIR model simplifications:** The discrete-time SIR model treats all CVE-CVE connections as equally likely transmission pathways. In reality, transmission depends on actual deployment topology, which varies by organization.
-

Technical Execution Note

The pipeline ran in **205 seconds** on a standard laptop. All computations were accelerated using the igraph C library for betweenness, closeness, and eigenvector centrality (replacing NetworkX's Python-only implementations which would have taken 30–60 minutes for the same tasks).

Report prepared by Group 8 | MITS 6700G | Network Science Final Project

Data source: NVD CVE 2021 via fkie-cad/nvd-json-data-feeds (Fraunhofer FKIE)

Analysis completed using Python 3.12, NetworkX 3.6.1, igraph 1.0.0